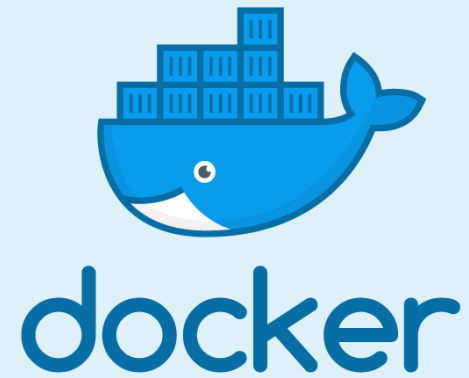




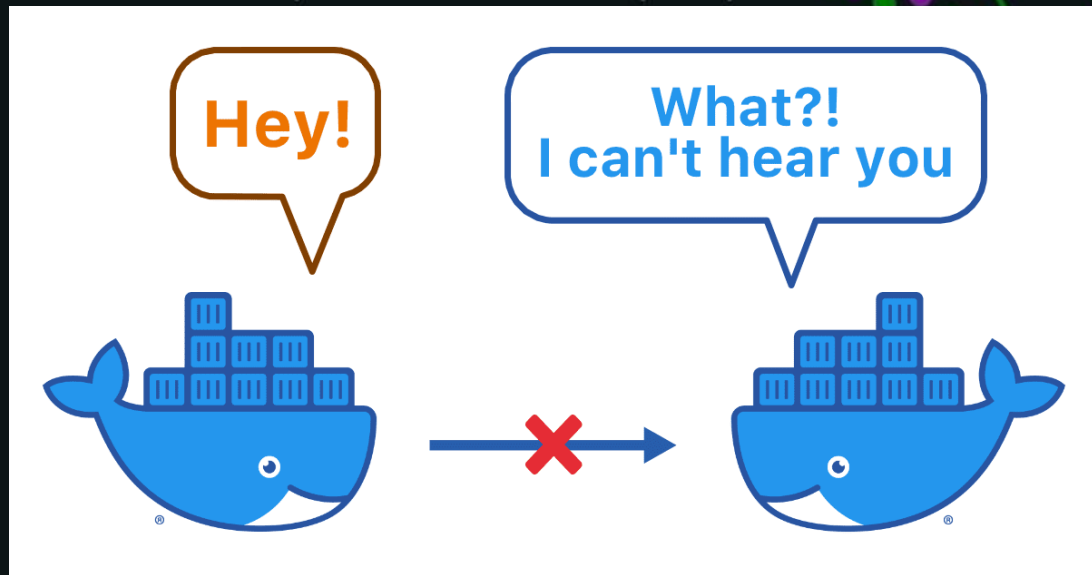
Composing

Using Docker Compose to allow containers to communicate

In the previous video...



In this video...



Why can't the containers communicate

- Docker isolates containers due to security and unintended interactions
- The isolation means that one container cannot automatically see or access another container's resources
- Network isolation:
 - If 2 containers are on different networks/ports, they cannot communicate directly
- There are ways to make them communicate through a designated network, the best way is through Docker Compose

What is Docker Compose?

- Tool used to define and manage multi-container Docker applications/images
- It enables communication between different images that are on different networks/ports.
- This is done through a single configuration file such that we don't use multiple Docker commands.

Why should we compose?

- Allows interconnected services to run as single stack
- A single command to start all the services
- Ensures development, testing and staging environments are identical

What exactly are we composing?

- Frontend Image:
 - Vue -> Node
 - Dependencies -> node_modules
 - Port 5173
- Backend Image:
 - Python v3.11
 - Dependencies -> Flask, Flask-cors, etc
 - Port 5000

Skeleton Code

```
# Version of Docker to use
version: '3.8'

# Services in the application
services:
  service1:
    build:
      context: . # use current directory
      dockerfile: <path> # path to the
Dockerfile
    ports:
      - "<host-port>:<container-port>" #
ports to be exposed + where to port it
to
    volumes: # Named volumes and paths
on the host mapped to paths in the
container
      - <path>
    command: <command to run> # The
command that will be run in the
container

  service2:
    build:
      context: .
      dockerfile: <path>
    image: <image1>
    container_name: <container1>
    ports:
      - "<host-port>:<container-port>"
    volumes:
      - <path>
    command: <command to run>
```


Skeleton Code

```
# Version of Docker to use
version: '3.8'

# Services in the application
services:
  service1:
    build:
      context: . # use current directory
      dockerfile: <path> # path to the
Dockerfile
    ports:
      - "<host-port>:<container-port>" #
ports to be exposed + where to port it
to
    volumes: # Named volumes and paths
on the host mapped to paths in the
container
      - <path>
    command: <command to run> # The
command that will be run in the
container

  service2:
    build:
      context: .
      dockerfile: <path>
    image: <image1>
    container_name: <container1>
    ports:
      - "<host-port>:<container-port>"
    volumes:
      - <path>
    command: <command to run>
```

Skeleton Code

```
# Version of Docker to use
version: '3.8'

# Services in the application
services:
  service1:
    build:
      context: . # use current directory
      dockerfile: <path> # path to the
Dockerfile
    ports:
      - "<host-port>:<container-port>" #
ports to be exposed + where to port it
to
    volumes: # Named volumes and paths
on the host mapped to paths in the
container
      - <path>
    command: <command to run> # The
command that will be run in the
container

  service2:
    build:
      context: .
      dockerfile: <path>
    image: <image1>
    container_name: <container1>
    ports:
      - "<host-port>:<container-port>"
    volumes:
      - <path>
    command: <command to run>
```

Skeleton Code

```
# Version of Docker to use
version: '3.8'

# Services in the application
services:
  service1:
    build:
      context: . # use current directory
      dockerfile: <path> # path to the
Dockerfile
      ports:
        - "<host-port>:<container-port>" #
ports to be exposed + where to port it
to

      volumes: # Named volumes and paths
on the host mapped to paths in the
container
        - <path>
      command: <command to run> # The
command that will be run in the
container

  service2:
    build:
      context: .
      dockerfile: <path>
    image: <image1>
    container_name: <container1>
    ports:
      - "<host-port>:<container-port>"
    volumes:
      - <path>
    command: <command to run>
```

Skeleton Code

```
# Version of Docker to use
version: '3.8'

# Services in the application
services:
  service1:
    build:
      context: . # use current directory
      dockerfile: <path> # path to the
Dockerfile
    ports:
      - "<host-port>:<container-port>" #
ports to be exposed + where to port it
to
      volumes: # Named volumes and paths
on the host mapped to paths in the
container
      - <path>
    command: <command to run> # The
command that will be run in the
container

  service2:
    build:
      context: .
      dockerfile: <path>
    image: <image1>
    container_name: <container1>
    ports:
      - "<host-port>:<container-port>"
    volumes:
      - <path>
    command: <command to run>
```

Skeleton Code

```
# Version of Docker to use
version: '3.8'

# Services in the application
services:
  service1:
    build:
      context: . # use current directory
      dockerfile: <path> # path to the
Dockerfile
    ports:
      - "<host-port>:<container-port>" #
ports to be exposed + where to port it
to
    volumes: # Named volumes and paths
on the host mapped to paths in the
container
      - <path>
    command: <command to run> # The
command that will be run in the
container

  service2:
    build:
      context: .
      dockerfile: <path>
    image: <image1>
    container_name: <container1>
    ports:
      - "<host-port>:<container-port>"
    volumes:
      - <path>
    command: <command to run>
```


Other sections

- Networks:
 - Custom networks for containers
- Configs:
 - Injecting configuration files
- Secrets:
 - Stores sensitive data
- env_file:
 - Load environment variables from a file
- Labels:
 - Metadata

Proposed compose file



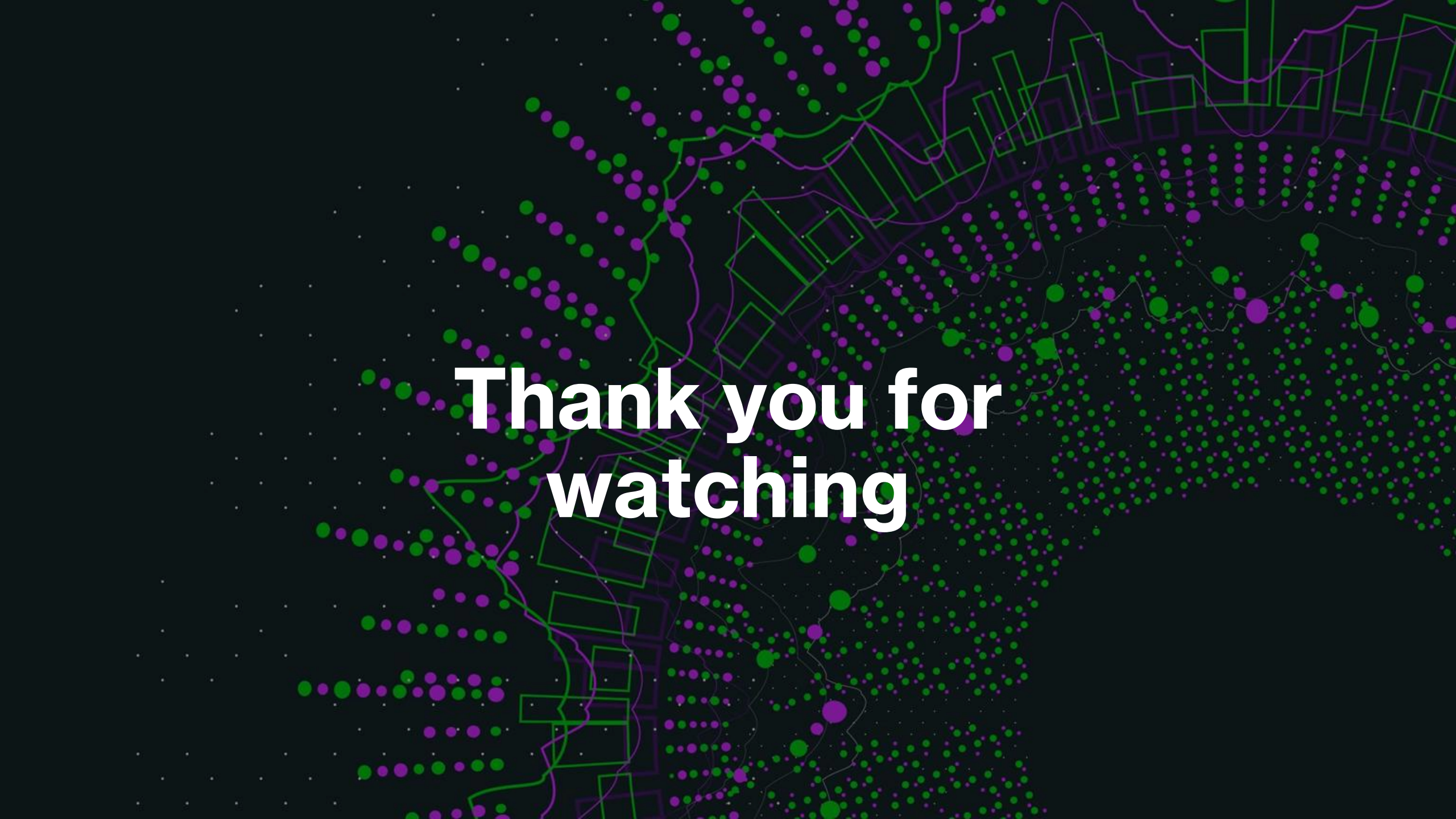
```
version: '3.9'

services:
  backend:
    build:
      context: .
      dockerfile: backend/Dockerfile
    ports:
      - "5000:5000"
    volumes:
      - ./app
    command: python -m backend.app

  frontend:
```

How to run a compose file

- A compose file is run using the command:
 - ``docker compose up``
- This single command will build each image.
- Once they are successfully built and started, compose will greenlight the stack and a user can click on the frontend port or open the port in their browser.

The background is a black field filled with a complex pattern of green and purple dots, lines, and rectangles. The dots are arranged in various clusters and trails. The lines are thin and wavy, connecting some of the dots. The rectangles are thin and appear to be floating or overlapping the other elements. The overall effect is a dense, abstract, and somewhat chaotic visual field.

**Thank you for
watching**